

ECET 230: - Electronics Design for Manufacturing and Production

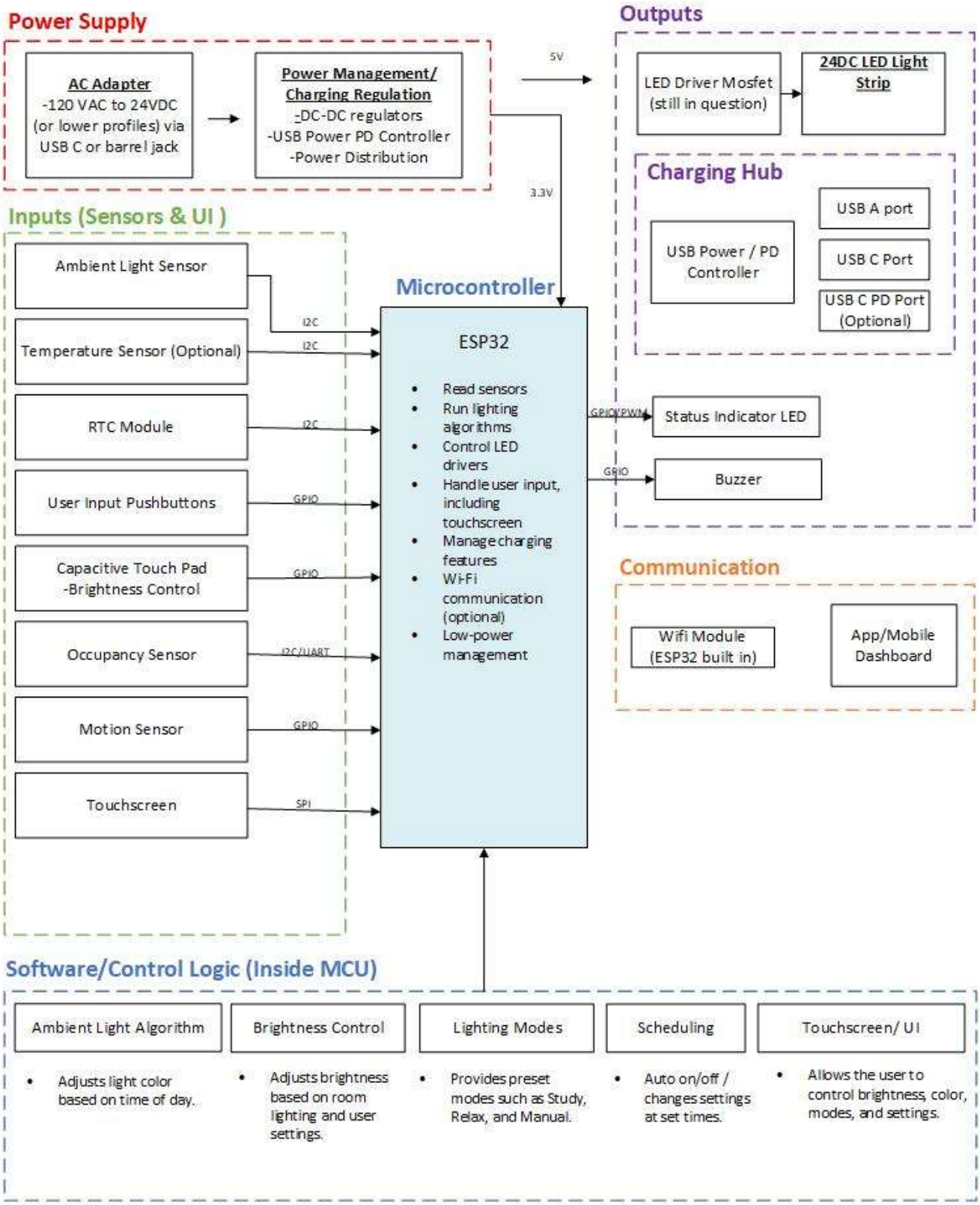
Major Component Selection

Professor Hansis

Group: Desk Beam, Multifunctional Study Lamp

Date: September 22, 2026

Block Diagram (Visio):



Bryan Carpio
ECET 230
Block Diagram

Electronic Components List (May be Subject to Change):

1. **Microcontroller — ESP32-S3**

Options Considered

- Arduino Uno, Raspberry Pi Pico W, ESP32, ESP32-S3

Selected Implementation

- ESP32-S3 DevKitC-1

Why We Chose It

- Built-in Wi-Fi.
- Supports GPIO, I^2C, SPI, UART, and PWM.
- Enough processing power for the touchscreen and lighting algorithms.
- Easy to breadboard and program through USB.
- Can later be replaced by an ESP32-S3 module on the final PCB.

Communication

- I^2C, SPI, UART, GPIO, and PWM depending on the connected device.

Breadboard/Test Implementation

- Connect the development board through USB.
- Run basic programs to test GPIO, I^2C, SPI, PWM, and Wi-Fi individually.

Purchase Link:
Amazon ESP32 Dev Board

2. Main Power Supply

Options Considered

- 12 V DC adapter
- 24 V DC adapter
- USB-C PD input
- Internal AC-DC supply

Selected Implementation

- External 120 VAC to 24 VDC power adapter

Why We Chose It

- Directly supports the 24 V LED lighting system.
- Keeps dangerous mains voltage outside of the prototype.
- 24 V allows relatively high power without excessive current.
- Provides one main voltage rail that can be stepped down for the electronics.

Breadboard/Test Implementation

- Use the external supply as the main 24 V rail.
- Verify the output voltage using a multimeter before attaching electronics.

Selected Prototype Supply

- Mean Well GST60A24-P1J
- 24 V
- 2.5 A
- 60 W

Purchase Link:
Mean Well 24 V 60 W Power Adapter — DigiKey

3. **DC-DC Voltage Regulation**

Options Considered

- Linear regulator, LDO , DC-DC buck converter

Selected Implementation

- 24 V → 5 V buck converter

Why We Chose It

- The LED system can remain on 24 V while the ESP32, touchscreen, and other electronics receive lower voltage.
- Much more efficient than dropping 24 V to 5 V through a linear regulator.
- Easy to prototype using a prebuilt module.

Breadboard/Test Implementation

- Apply 24 V to the converter.
- Measure 5 V output.
- Connect a small load and make sure the voltage remains stable.

Purchase Link:
Buck Converter Amazon

4. **Ambient Light Sensor**

Options Considered

- LDR/photoresistor, BH1750, VEML7700

Selected Implementation

- VEML7700 digital ambient-light sensor

Why We Chose It

- Measures illumination in lux rather than giving only a basic analog value.
- I^2C interface simplifies wiring.
- Allows the lamp to automatically increase or decrease brightness based on the room.

Communication

- I^2C

Breadboard/Test Implementation

- Connect SDA, SCL, power, and ground to ESP32.
- Print lux readings to the Serial Monitor.
- Test by covering the sensor or changing room lighting.

Purchase Link:

VEML7700 Amazon

5. **Temperature/Humidity Sensor**

Options Considered

- DHT11, DHT22, AHT20/AHT21, BME280

Selected Implementation

- AHT20/AHT21

Why We Chose It

- Small and inexpensive. Measures both temperature and humidity.
- Uses I^2C, so it can share the same bus with the ambient-light sensor and RTC.
- Straightforward Arduino/ESP32 libraries are available. Adafruit's AHT20 breakout supports 3.3 V systems and I^2C communication.

Communication

- I^2C

Breadboard/Test Implementation

- Read temperature and humidity every few seconds.
- Display results through Serial Monitor.

Purchase Link:

AHT20 Amazon

6. **Real-Time Clock**

Options Considered

- ESP32 software clock, Internet/NTP time only, DS1307, DS3231

Selected Implementation

- DS3231 RTC

Why We Chose It

- Keeps time even when Wi-Fi is unavailable.
- Battery backup allows it to retain the time when the lamp is unplugged.
- More accurate than many basic RTC modules.
- Uses I^2C.

Communication

- I^2C

Breadboard/Test Implementation

- Set the date/time once.
- Unplug and reconnect the ESP32.
- Verify that the RTC maintained the correct time.

Purchase Link:

DS3231 RTC Module

7. User Pushbuttons

Options Considered

- Mechanical pushbuttons

Selected Implementation

- Momentary pushbuttons for basic physical controls

Why We Chose It

- Extremely simple and reliable.

Communication

- GPIO

Breadboard/Test Implementation

- Connect button between GPIO and ground using the ESP32's pull-up resistor.
- Print whether the button is pressed or released.

Purchase Link:

- Basic pushbuttons (can be found anywhere including the makerspace).

8. **Capacitive Touch Pad**

Options Considered

- Mechanical button

Selected Implementation

- TTP223 capacitive-touch module

Why We Chose It

- Provides a clean touch-sensitive control without a moving button.
- Can be mounted underneath thin plastic in the final lamp.

Communication

- GPIO

Breadboard/Test Implementation

- ESP32 detects HIGH/LOW.
- Use the touch input to increase or decrease a test LED's brightness.

Purchase Link:
TTP223 Capacitive Touch Amazon

9. **Occupancy Sensor**

Options Considered

- PIR motion sensor
- Ultrasonic sensor
- 24 GHz mmWave presence sensor

Selected Implementation

- **DFRobot C4001 24 GHz mmWave presence sensor**

Why We Chose It

- A PIR is good at detecting someone moving, but a student may sit almost completely still while studying.
- mmWave can detect both movement and small human motions associated with someone remaining present.
- The C4001 supports I^2C and UART and is compatible with ESP32.

Communication

- I^2C or UART
- I would use UART so your I^2C sensor bus stays less crowded.

Breadboard/Test Implementation

- Display "OCCUPIED" when presence is detected.
- Display "EMPTY" when no person is detected.

Purchase Link:

Occupancy Sensor

10. **Motion Sensor**

Options Considered

- PIR
- Ultrasonic
- mmWave

Selected Implementation

- PIR motion sensor

Why We Chose It

- Very inexpensive.
- Simple digital output.
- Useful for detecting someone approaching the desk.
- Complements the mmWave occupancy sensor rather than performing exactly the same task.

The breadboard-friendly Adafruit PIR operates from 3–12 V and produces a 3.3 V HIGH/LOW output, making it convenient for an ESP32.

Communication

- GPIO

Breadboard/Test Implementation

- Turn an LED on whenever motion is detected.

Purchase Link: https://www.adafruit.com/product/4871?

11. Touchscreen Display

Options Considered

- OLED display + buttons
- LCD without touch

Selected Prototype Implementation

- 2.8-inch ILI9341 TFT touchscreen

Why We Chose It

- Provides enough space for brightness, color temperature, scheduling, and mode controls.
- Supports SPI communication.
- Available as a breadboard-friendly breakout.
- 320 × 240 resolution.
- Includes touchscreen capability.

Communication

- Primarily SPI

Breadboard/Test Implementation

- Display buttons such as:
    - Brightness +
    - Brightness −
    - Study Mode
    - Relax Mode
- Print the selected option when the screen is touched.

Purchase Link:
Adafruit 2.8-inch ILI9341 Touchscreen

12. Main Led Light

Options Considered

- Single high-power LED
- Standard SMD LED strip
- RGB strip

Selected Implementation

- 24 V tunable-white COB LED strip

Why We Chose It

- COB gives you a more continuous line of light rather than visible LED dots.
- 24 V works directly from the lamp's main power rail.
- Tunable white provides separate warm and cool LED channels.
- Allows the lamp to change color temperature throughout the day.
- Can be cut to the required lamp length.

The Armacost option is a 24 V tunable-white COB strip specifically designed to vary white color temperature.

Breadboard/Test Implementation

- Initially power a short section directly at 24 V.
- Then connect each lighting channel to its MOSFET control circuit

LED Strip

13. Mosfet LED Control

Options Considered

- Relay
- Constant-current driver
- MOSFET switching
- LED controller

Selected Implementation

- **Logic-level MOSFET PWM control**

Why We Chose It

- The 24 V LED strip already has a regulated power source.
- The ESP32 cannot power the strip directly.
- MOSFETs allow the ESP32 to control brightness using PWM.
- Two channels control the warm and cool LEDs separately.

Communication

- PWM from ESP32 GPIO

Test

- Change PWM duty cycle from 0–100% and observe brightness.

Prototype Module:
Adafruit MOSFET Driver Module

USB A-USB C:
https://www.amazon.com/s?k=IP6518+USB+C+PD+charging+module